

Быстрый рендеринг с DOM шаблонизаторами

Каплуновский Борис
@bskaplou
bk@aviasales.ru



aviasales.ru

Agenda

- Правила игры
- Типы шаблонизаторов
- Минусы AngularJS/ReactJS
- Плюсы менее известных библиотек
- Шаблонизатор temple



Правила игры



- UX зависит от:
 - Скорости инициализации страницы
 - Скорости реакции интерфейса
 - Потребления ресурсов (gc)
 - Размера библиотеки

Строковые шаблонизаторы

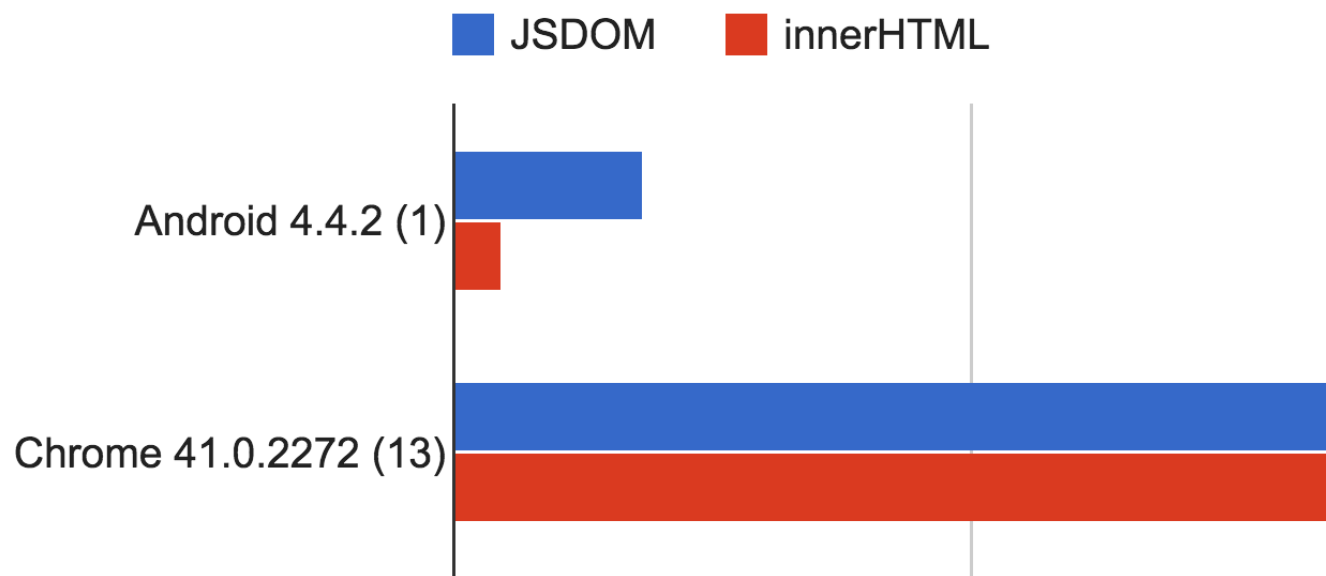
doT, Handlebars и другие

- При каждом изменении данных перестраивается значительный кусок DOM
- Использованный участок DOM выбрасывается
- Не используются якоря для обновления данных

DOM API



DOM API быстрее на мобильных устройствах



<http://jsperf.com/innerHTML-vs-domjs>

Создание DOM через API

- Много скучного кода → создание одного элемента 5-10 строк кода

```
document.createElement
```

```
document.createTextNode
```

```
element.appendChild
```

```
element.setAttribute
```

```
textNode.nodeValue
```

DOM шаблонизаторы

- Можно сохранить DOM объекты в переменных для обновления данных
- Можно использовать один участок DOM несколько раз

Где оступились гиганты

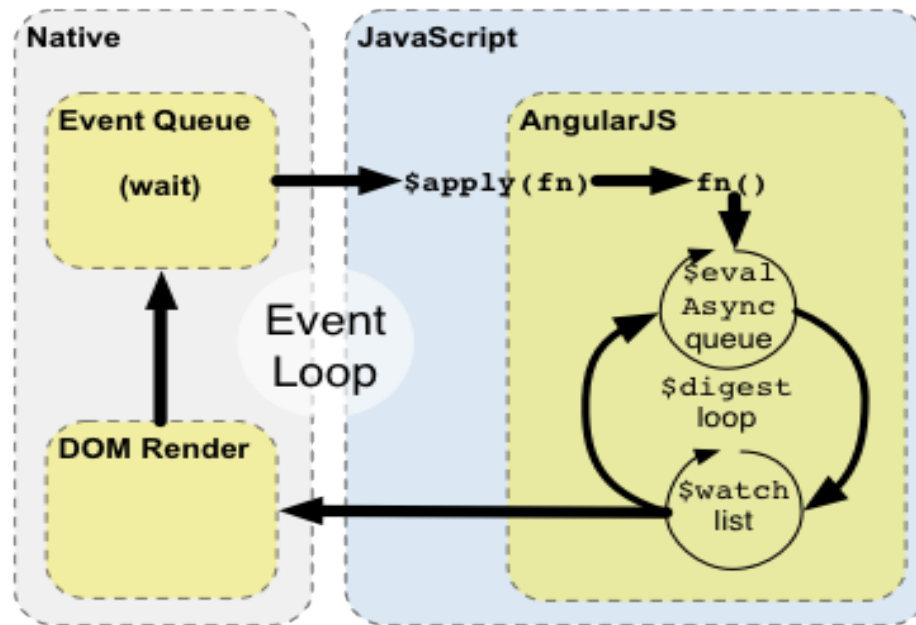


Почему тормозит AngularJS



- Компиляция шаблона из DOM или строки на клиенте
 - Чем больше шаблонов тем медленнее загрузка

Почему тормозит AngularJS



- \$watcher плодятся и если за ними не следить, бьют по производительности

Почему тормозит AngularJS

- Работа с DOM разбросана по директивам
- Отсутствие true way способа работы с DOM
- ~50kb min gz размер библиотеки

Почему тормозит ReactJS

- Кроме DOM в памяти хранятся ещё 2 копии VirtualDOM
- На каждое изменение данных создаётся ещё одна копия VirtualDOM
- ~35kb min gz размер библиотеки

ReactJS ничего не знает о семантике данных

```
<div>  
  <div>  
    <i>{{VALUE}}</i>  
  </div>  
</div>
```

- Чтобы обновить VALUE VirtualDOM должен сравнить 3 элемента и одну текстовую ноду

А теперь кое-что другое



RiotJS – правильный AngularJS

- Правильный размер 5kb min gz
- Шаблоны используют семантику данных
- Компиляция шаблонов всё ещё происходит на клиенте

PaperclipJS

- Использует cloneNode для создания быстрого создания копии участков DOM
- ~40kb min gz размер библиотеки
- Всё ещё beta

<http://paperclipjs.com/>

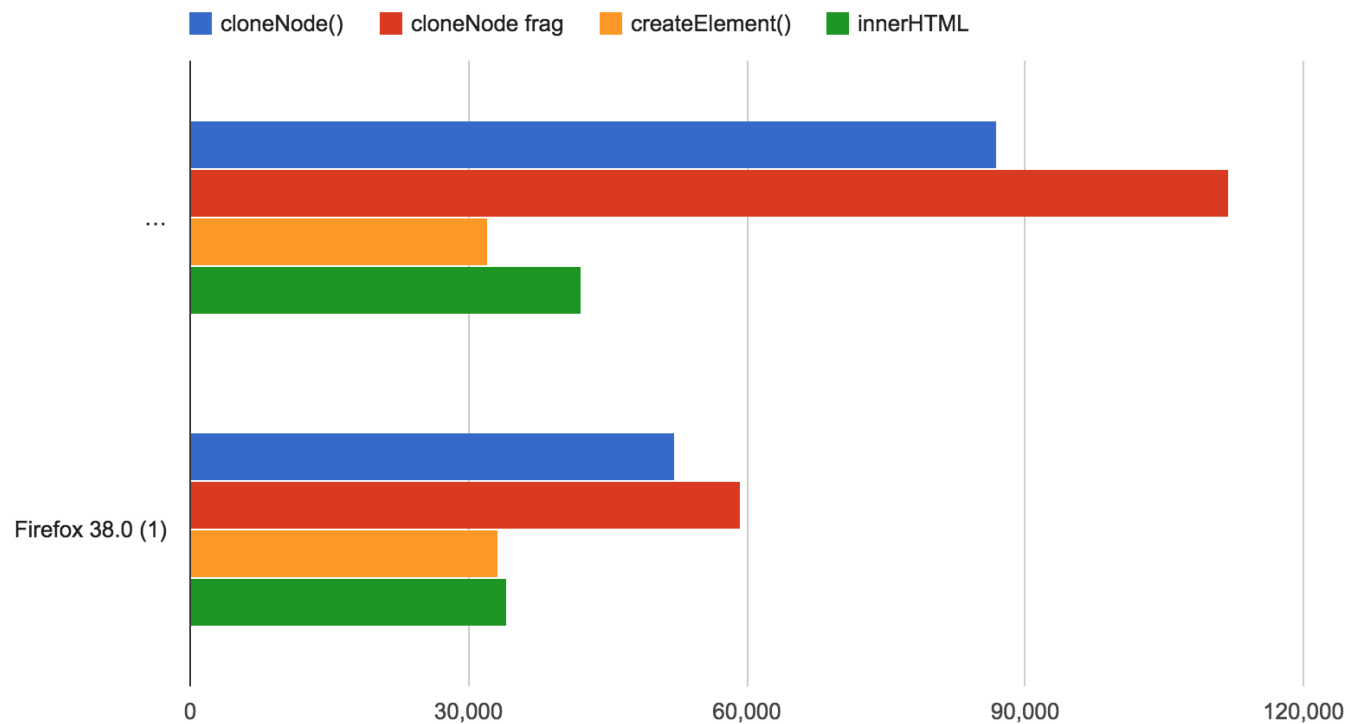
Техники позволяющие ускорить работу с DOM



VirtualDOM

- Плюсы:
 - Позволяет сократить вызовы к DOM
- Минусы
 - Overhead на создание копии DOM
 - Сложный алгоритм сравнения
 - Алгоритм ничего не знает о структуре данных

cloneNode & DocumentFragment



<http://jsperf.com/clonenode-vs-createelement-performance/55>

Создание DOM участков заранее

- REST запрос на сервер ~20ms
- Создание DOM под данные
- Разбор ответа сервера
- Вставка данных в готовый DOM
- Добавление DOM в страницу

Простой шаблонизатор TEMPLE

temple

- Шаблоны компилируются в JavaScript в момент сборки приложения
- Возможность создавать куски DOM заранее
- Переиспользование шаблонов
- Скромный размер библиотеки 700b min gz
- Максимально быстрое обновление DOM

Temple шаблон

```
<div class="{{div_classes}}">  
  <span class="{{span_classes}}">  
    {{value}}  
  </span>  
</div>
```


Temple шаблон

- `<forall key="k">` для циклов
- `<if key="k">` для ветвлений
- Не поддерживает выражения
 - Используйте Presenter или ViewModel для адаптации данных

Создание шаблона

```
var item = pool.get("item")  
item.update(data);  
parent.appendChild(item.root());
```

- Просто интегрируется в приложение

```
item.remove();
```

Заблаговременное создание DOM

```
pool.build_cache({“item”: 10});
```

- После этого вызова получение DOM для шаблона `item` происходит мгновенно
- Кешировать шаблоны можно ожидая ответа сервера или `DOMContentLoaded`

Быстрое внесение изменений в DOM

```
item.update({“value”: 10});
```

Просто делает

```
e0.nodeValue = 10; //глубина стека 2
```

Но можно ещё быстрее

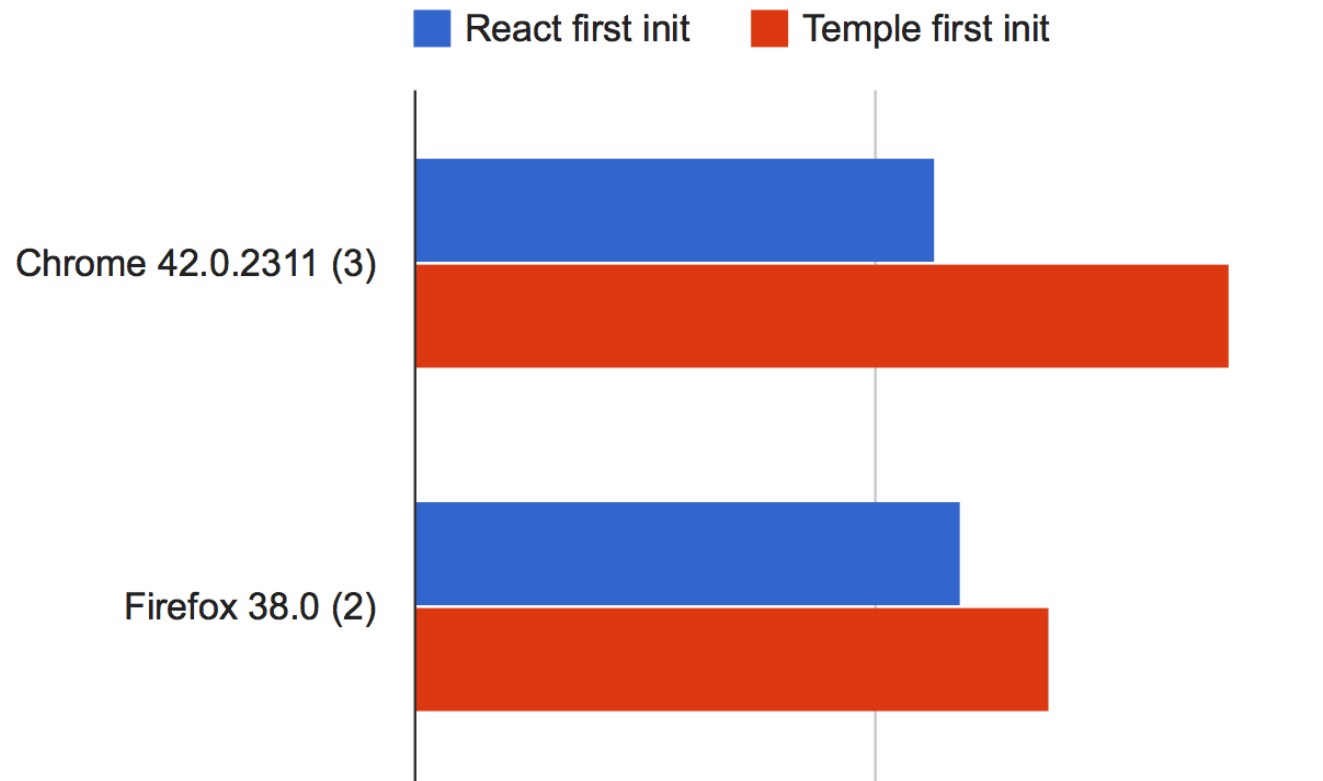
```
item.value(10); //глубина стека 1
```

Переиспользование шаблонов

```
pool.release("item", item);
```

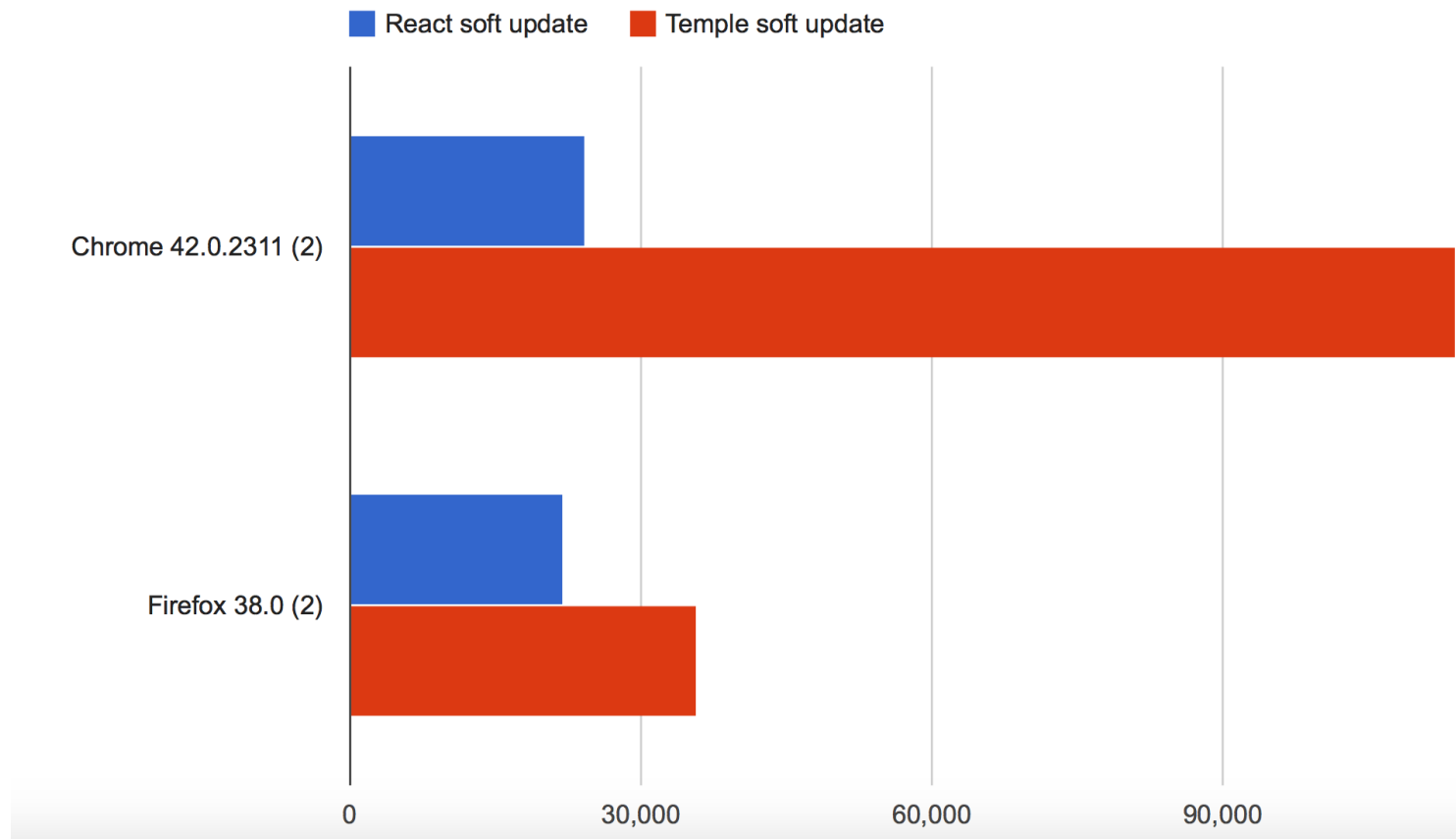
Этот же шаблон будет использован
позднее

Benchmarks INIT



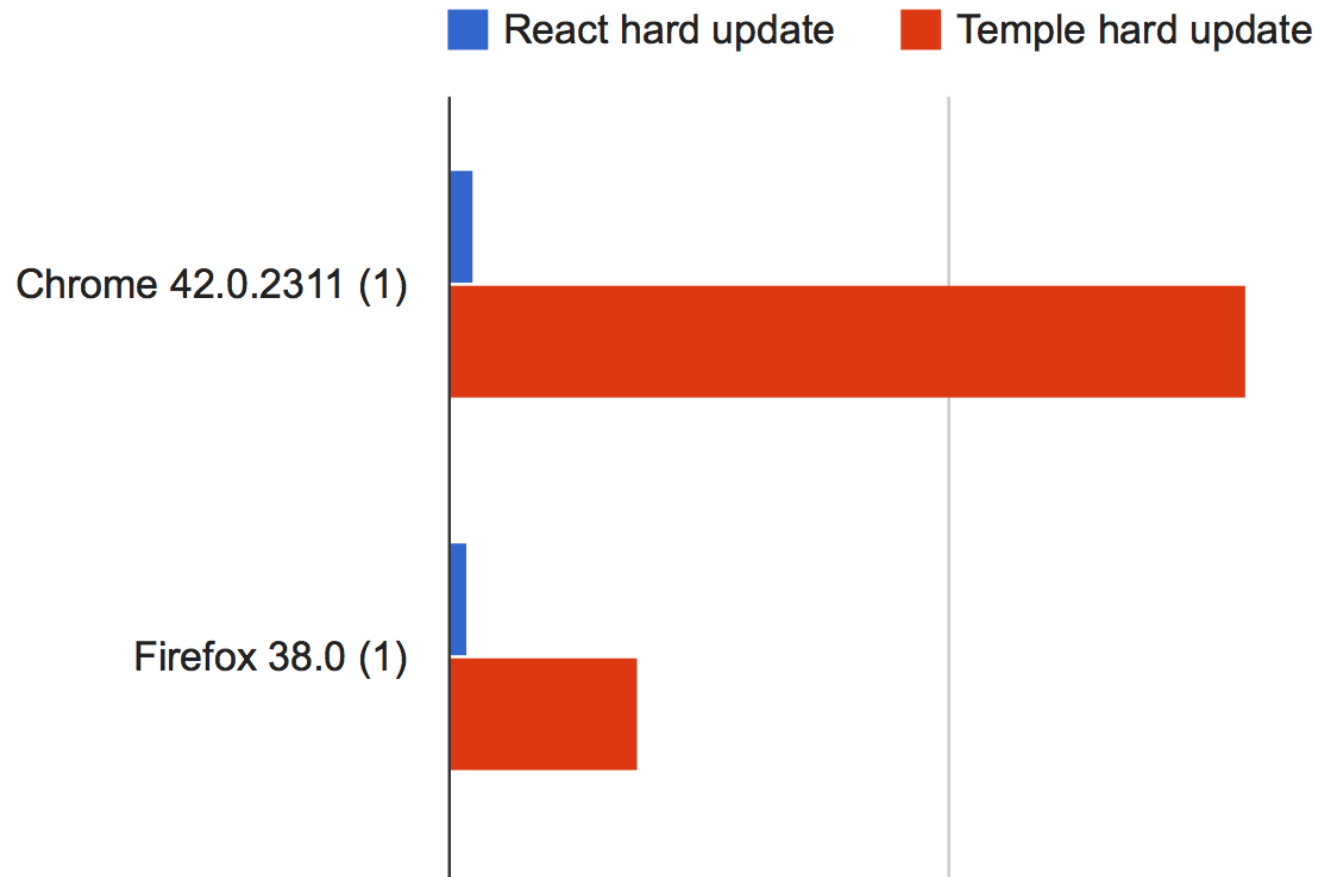
<http://jsperf.com/init-temple-vs-react>

Benchmarks soft UPDATE



<http://jsperf.com/soft-update-temple-vs-react>

Benchmarks hard UPDATE



<http://jsperf.com/hard-update-temple-vs-react>

Temple не работает с событиями

Используйте:

- Стандартные события
- Библиотеки для работы с событиями
- Мы используем domdelegate от Ftlabs
<https://github.com/ftlabs/ftdomdelegate>

Temple успешно работает

m.aviasales.ru

- 10kb шаблоны
- 58kb всё приложение

d.search.aviasales.ru

- 15kb шаблоны
- 70kb всё приложение

Fork Me

<https://github.com/KosyanMedia/template>

Всё ещё сыро но:

- Есть примеры и документация
- Есть плагины для gulp и grunt
- Хорошая производительность

Q & A

